



UNIVERSITÉ SIDI MOHAMED BEN ABDELLAH FACULTÉ DES SCIENCES ET TECHNIQUES FES

« Ingénierie Electrique et Systèmes Embarqués »

COMPTE RENDU: Électronique Numérique Avancée

Préparé par :

FRANKLIN HAMUNYEMBA
AMANSOUR MOHAMMED

Supervisé par :

Pr. RAZI

Introduction Générale

L'électronique numérique moderne repose de plus en plus sur l'utilisation de circuits logiques programmables (**FPGA**), qui offrent une flexibilité et une puissance de calcul indispensables aux systèmes embarqués actuels. Dans le cadre du module "Électronique Numérique Avancée", nous avons réalisé une série de travaux pratiques visant à maîtriser le flux de conception complet de ces systèmes, de la spécification théorique jusqu'à la réalisation matérielle.

L'objectif principal de ces séances était de nous familiariser avec le langage de description matérielle **VHDL** et les outils de CAO standards de l'industrie, notamment l'environnement **Quartus II** pour la synthèse et **ModelSim-Altera** pour la simulation comportementale.

Notre démarche a été progressive, nous permettant d'aborder des concepts de complexité croissante :

- La logique combinatoire simple (Portes logiques).
- La logique séquentielle et la gestion du temps (Compteurs, diviseurs de fréquence).
- La modélisation de systèmes complexes via des Machines à États Finis (FSM).

Ce compte rendu s'articule autour de deux parties distinctes :

1. **La première partie** est consacrée à la conception et à l'implémentation physique sur la carte de développement **DE2-115** (équipée d'un FPGA Cyclone IV E). Nous y détaillons la synthèse des circuits et la vérification réelle par interaction avec les périphériques de la carte (LEDs, afficheurs, boutons).
2. **La seconde partie** se concentre sur la simulation comportementale sous **ModelSim**. À travers l'exemple d'un registre à décalage, nous analyserons l'importance de valider temporellement la logique d'un circuit avant son intégration matérielle.

Table des matières

QUARTUS II.....	5
CONFIGURATION DE L'ENVIRONNEMENT	7
1. Installation du pilote USB-Blaster	7
2. Création d'un Nouveau Projet	9
3. Vérification de la Chaîne de Compilation.....	11
4. Accès au Programmeur	12
I. Manipulation 1 : Porte Logique XOR	13
a) Conception et Explication du Code VHDL.....	13
b) Affectation des Broches (Pin Planner).....	14
c) Programmation.....	14
II. Manipulation 2 : Clignotement de LED	15
a) Conception et Explication du Code VHDL.....	15
b) Affectation des Broches (Pin Planner).....	16
c) Programmation.....	17
III. Manipulation 3 : Le Chenillard	18
a) Conception et Explication du Code VHDL.....	18
b) Affectation des Broches (Pin Planner).....	19
c) Programmation.....	20
IV. Manipulation 4 : Compteur/Décompteur 7-Segments	21
a) Conception et Explication du Code VHDL.....	21
b) Affectation des Broches (Pin Planner).....	23
c) Programmation.....	24
V. Manipulation 5 : FSM - Gestion d'un Carrefour.....	25
a) Conception et Explication du Code VHDL.....	25
1. Ajustement de l'Horloge	25
2. Machine à États	26
3. Gestion Piéton	26
4. Le code final	26

b) Affectation des Broches (Pin Planner).....	29
c) Programmation.....	30
MODELSIM	32
Manipulation Unique : Registre à Décalage 4-bits (PISO).....	34
1. Configuration de l'Environnement et du Projet	34
a. Création du Projet de Simulation	34
b. Création et Ajout des Fichiers VHDL	35
2. Compilation	36
3. Analyse du Code VHDL	36
4. Analyse du Testbench	37
5. Résultats de Simulation	39

PREMIÈRE PARTIE

CONCEPTION ET SYNTHÈSE SOUS QUARTUS II

&

IMPLÉMENTATION MATÉRIELLE SUR FPGA



Introduction

Ces séances de travaux pratiques avaient pour objectif l'apprentissage du flux de conception sur FPGA à l'aide du langage **VHDL**. Nous avons progressé de la création de fonctions logiques de base vers des systèmes séquentiels complexes, en mettant l'accent sur la gestion des horloges et les machines à états finis (FSM) pour des applications réelles comme la gestion d'un carrefour routier.

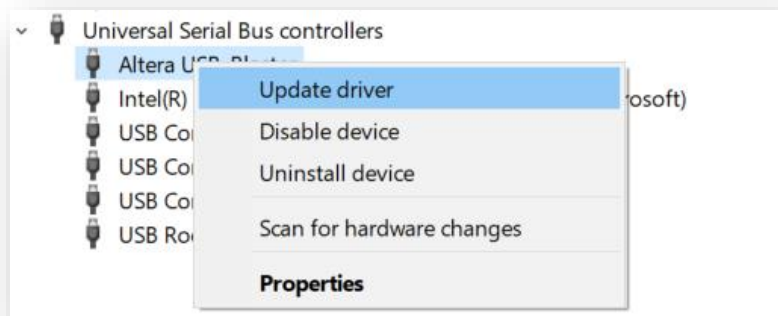
CONFIGURATION DE L'ENVIRONNEMENT

Pour réussir l'installation et la configuration de l'environnement de travail sous **Quartus II**, nous avons suivi une procédure rigoureuse permettant de l'interfacer avec la carte **FPGA Cyclone IV E**. Voici les étapes clés de cette configuration :

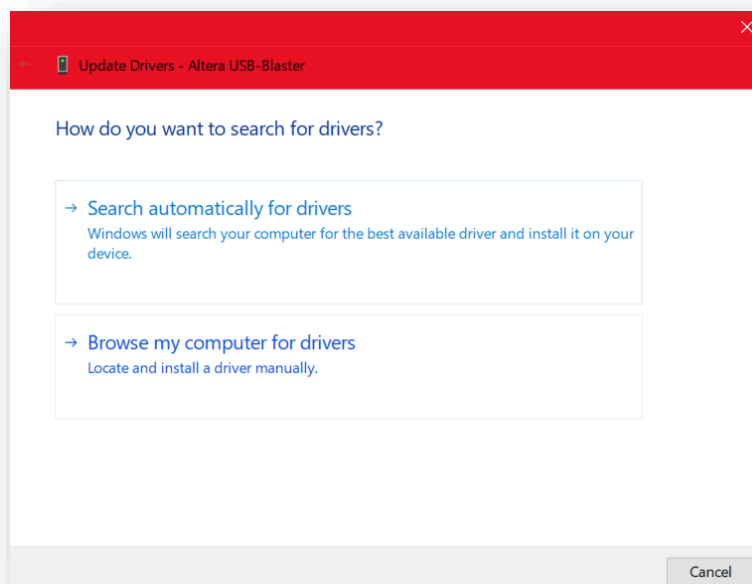
1. Installation du pilote USB-Blaster

C'est l'étape cruciale pour permettre au logiciel de communiquer avec le matériel.

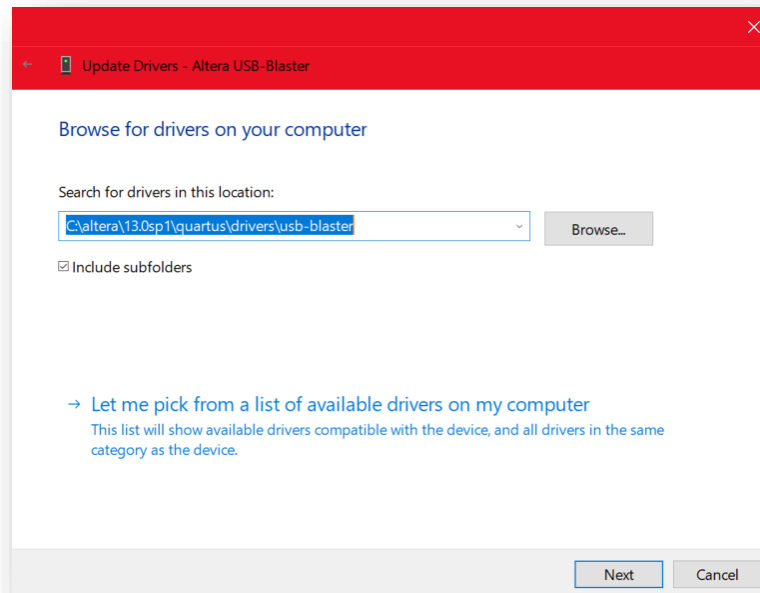
- Nous avons ouvert le **Gestionnaire de périphériques** de Windows pour localiser le périphérique "Altera USB-Blaster" non reconnu.



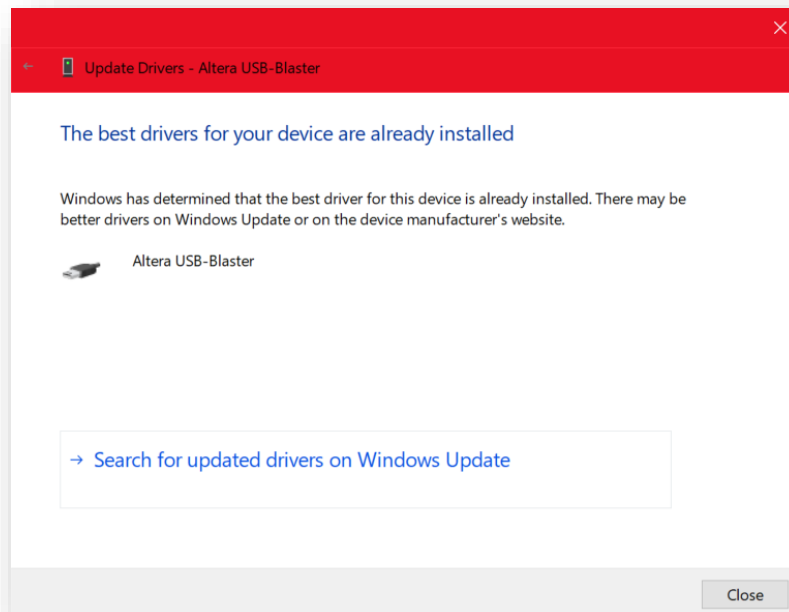
- En choisissant "Mettre à jour le pilote", nous avons sélectionné l'option de recherche manuelle sur l'ordinateur.



- Le chemin d'accès utilisé était le répertoire d'installation de Quartus : *C:\altera\13.0sp1\quartus\drivers\usb-blaster*.

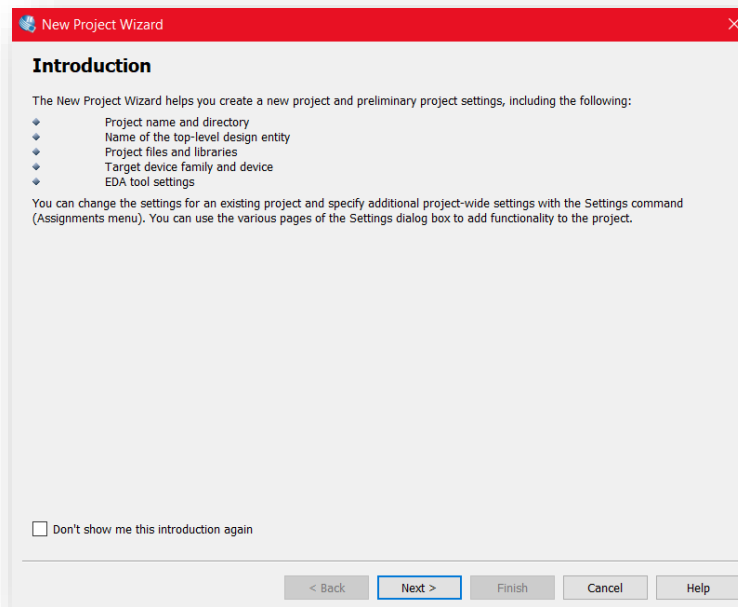


- Une fois l'installation validée, Windows confirme que "Le meilleur pilote pour votre appareil est déjà installé".

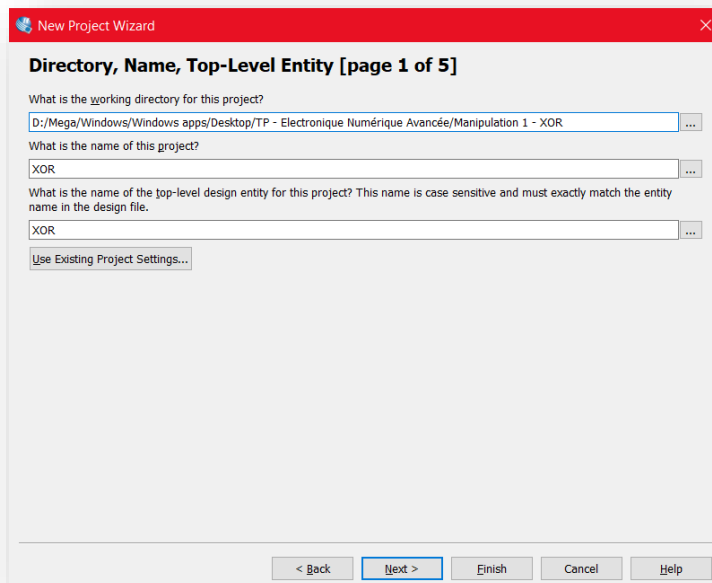


2. Création d'un Nouveau Projet

Pour chaque manipulation, nous avons utilisé l'assistant de création de projet pour garantir la compatibilité avec la cible.



- **Répertoire et Nom** : Nous avons défini un dossier spécifique (ex: Manipulation 1 - XOR) et nommé le projet ainsi que l'entité de haut niveau de manière identique (ex: XOR).



- **Sélection du Composant** : Dans les paramètres "Family & Device", nous avons impérativement sélectionné la famille **Cyclone IV E** et le modèle précis présent sur notre carte : **EP4CE115F29C7**.

Family & Device Settings [page 3 of 5]

Select the family and device you want to target for compilation.
You can install additional device support with the Install Devices command on the Tools menu.

Device family: Family: **Cyclone IV E** Devices: **All**

Target device:
☐ Auto device selected by the Fitter
☒ Specific device selected in 'Available devices' list
☐ Other: n/a

Show in 'Available devices' list:
 Package: **Any**
 Pin count: **Any**
 Speed grade: **Any**
 Name filter:
☒ Show advanced devices ☐ HardCopy compatible only

Available devices:

Name	Core Voltage	LEs	User I/Os	Memory Bits	Embedded multi
EP4CE115F29C7	1.2V	114480	529	3981312	532
EP4CE115F29C8	1.2V	114480	529	3981312	532

Companion device:
 HardCopy:
☐ Limit DSP & RAM to HardCopy device resources

< Back **Next >** Finish Cancel Help

- **Paramètres EDA** : Pour ces TPs, les outils de simulation et de synthèse ont été laissés sur <None>, utilisant uniquement les outils natifs de Quartus II.

EDA Tool Settings [page 4 of 5]

Specify the other EDA tools used with the Quartus II software to develop your project.

EDA tools:

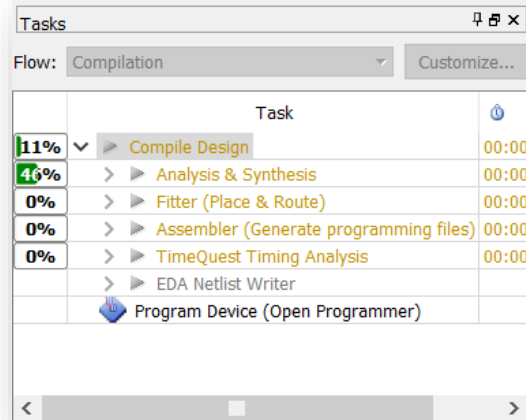
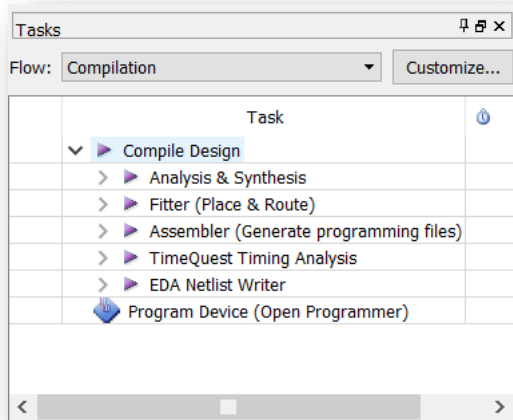
Tool Type	Tool Name	Format(s)	Run Tool Automatically
Design Entry/Synt...	<None>	<None>	☒ Run this tool automatically to synthesize the current design
Simulation	<None>	<None>	☒ Run gate-level simulation automatically after compilation
Formal Verification	<None>	<None>	
Board-Level	Timing	<None>	
	Symbol	<None>	
	Signal Integrity	<None>	
	Boundary Scan	<None>	

< Back **Next >** Finish Cancel Help

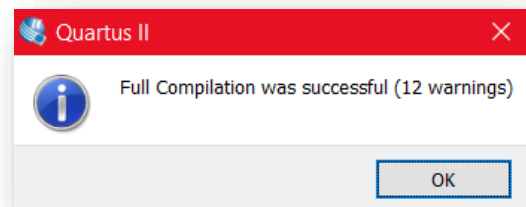
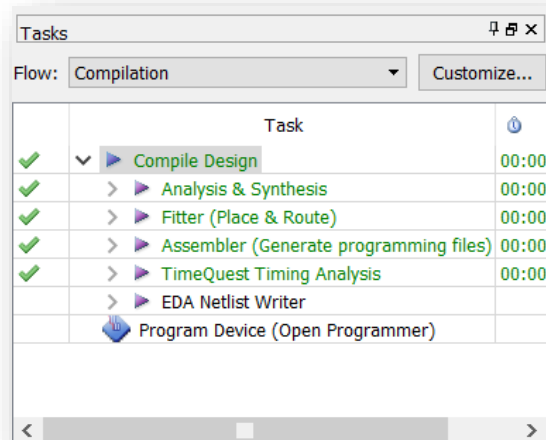
3. Vérification de la Chaîne de Compilation

Une fois le code écrit, la configuration se termine par une compilation d'essai pour valider les paramètres.

- L'utilisation de la fenêtre **Tasks** permet de lancer le flux complet : *Analysis & Synthesis*, *Fitter*, *Assembler*, et *TimeQuest Timing Analysis*.

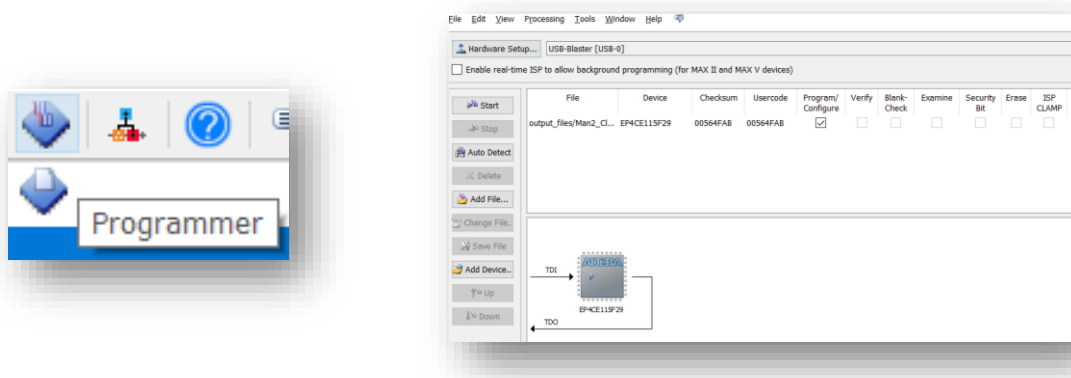


- Une configuration réussie se traduit par un message "Full Compilation was successful" avec l'indication du nombre d'avertissements (souvent liés aux contraintes de timing non définies).

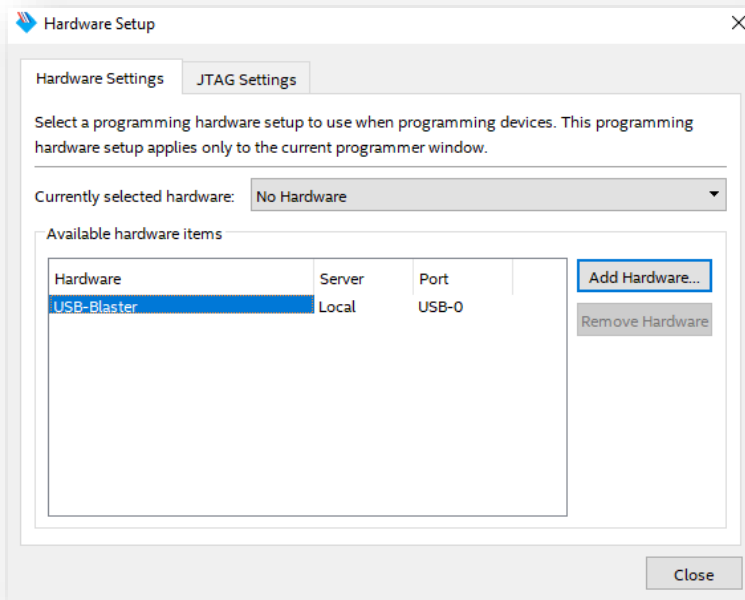


4. Accès au Programmeur

Enfin, nous avons configuré le Programmer pour lier le fichier `.sof` généré à la carte.



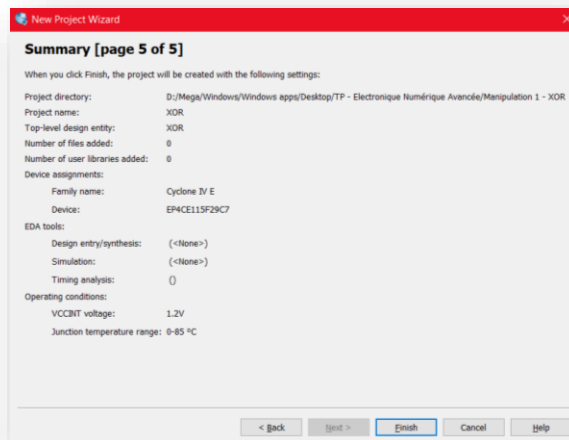
- Il faut s'assurer que le "Hardware Setup" affiche bien USB-Blaster [USB-0].



I. Manipulation 1 : Porte Logique XOR

L'objectif de cette première manipulation était de se familiariser avec le flux de conception de base sous **Quartus II**. Nous avons implémenté une fonction logique combinatoire simple, la porte **OU exclusif (XOR)**, afin de vérifier la chaîne complète : de l'écriture du code VHDL jusqu'à la programmation sur la carte FPGA **Cyclone IV E**.

Nous avons créé un nouveau projet sur lequel travailler, comme résumé dans l'image ci-dessous :



a) Conception et Explication du Code VHDL

Pour cette manipulation, nous avons utilisé le code suivant :

```
1. library ieee;
2. use ieee.std_logic_1164.all;
3.
4. entity Man1_XOR is
5. port(a,b : in std_logic;
6.      s : out std_logic
7. );
8. end;
9.
10. architecture behavior of Man1_XOR is
11. begin
12.     s <= a xor b;
13. end;
```

Entité : Elle définit l'interface matérielle du circuit. Nous avons deux entrées a et b (représentant les interrupteurs) et une sortie s (représentant une LED).

Architecture : Elle décrit le fonctionnement interne. L'instruction $s \leq a \text{ xor } b$; définit une logique combinatoire où la sortie est à '1' uniquement si une seule des deux entrées est active.

Une fois le code saisi, nous avons lancé le processus de Full Compilation. Cette étape transforme le code VHDL en une structure de portes logiques réelles (Netlist) adaptée au FPGA.

b) Affectation des Broches (Pin Planner)

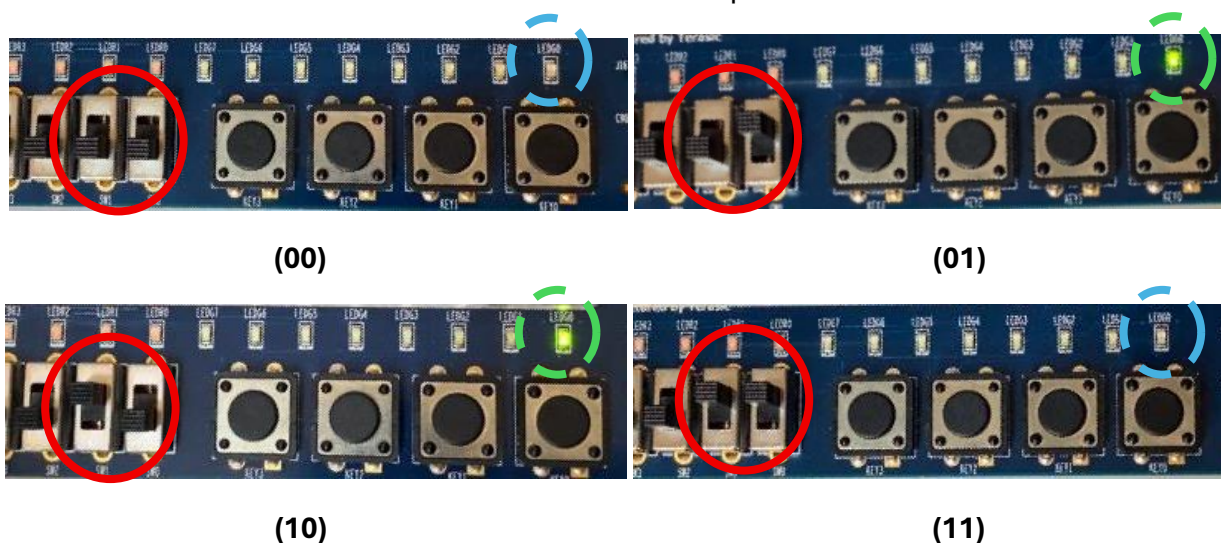
Pour que le circuit fonctionne physiquement sur la carte, nous avons dû lier les signaux du code aux composants de la carte (interrupteurs et LEDs) via le **Pin Planner**.

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair
in_a	Input	PIN_AB28	5	B5_N1	PIN_AB5	2.5 V (default)		8mA (default)		
in_b	Input	PIN_AC28	5	B5_N2	PIN_AB6	2.5 V (default)		8mA (default)		
out_s	Output	PIN_E21	7	B7_N0	PIN_AC4	2.5 V (default)		8mA (default)	2 (default)	

Nom du Signal	Type	Broche (Pin)	Composant Physique
a	Entrée	PIN_AB28	Interrupteur
b	Entrée	PIN_AC28	Interrupteur
s	Sortie	PIN_E21	Diode LED

c) Programmation

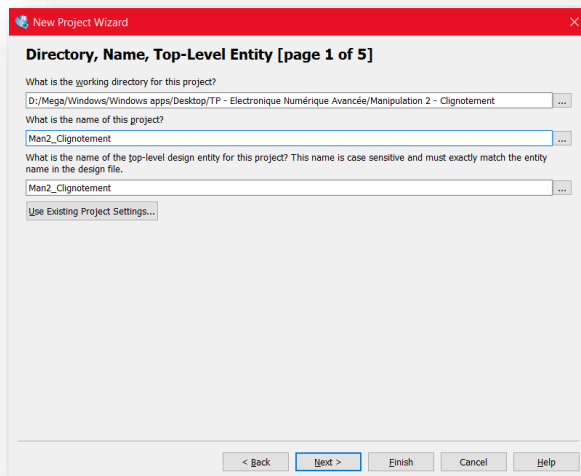
La dernière étape a consisté à ouvrir le **Programmer**. Nous avons vérifié que le matériel **USB-Blaster** était détecté, puis nous avons chargé le fichier de programmation .sof dans le FPGA. En manipulant les interrupteurs AB28 et AC28, nous avons pu observer l'allumage de la LED E21 conformément à la table de vérité de la porte XOR.



II. Manipulation 2 : Clignotement de LED

L'objectif de cette manipulation était de faire clignoter une LED à une fréquence visible à l'œil nu. L'horloge du FPGA fonctionnant à 50 MHz, une connexion directe résulterait en un clignotement imperceptible. Nous avons donc implémenté un diviseur de fréquence utilisant un compteur interne pour ralentir le signal.

Nous avons créé un nouveau projet nommé Man2_Clignotement, comme résumé dans l'image ci-dessous :



a) Conception et Explication du Code VHDL

Pour cette manipulation, nous avons utilisé le code suivant :

```
1. library ieee;
2. use ieee.std_logic_1164.all;
3. use ieee.std_logic_unsigned.all;
4. use ieee.numeric_std.all;
5.
6. entity Man2_Clignotement is
7. port(
8.     clk_50, reset : in std_logic;
9.     s : out std_logic
10. );
11. end;
12.
13. architecture behavior of Man2_Clignotement is
14. signal x , clk: std_logic := '0';
15. signal timer : std_logic_vector(23 downto 0);
16.
```

```

17. begin
18. process(clk_50, reset)
19.     begin
20.         if reset = '0' then
21.             timer <= (others => '0');
22.         elsif rising_edge(clk_50) then
23.             timer <= timer + 1;
24.         end if;
25.     end process;
26.     clk <= timer(23);
27.
28. process(clk, reset)
29.     begin
30.         if reset = '0' then
31.             x <= '1';
32.         elsif rising_edge(clk) then
33.             x <= not(x);
34.         end if;
35.     end process;
36.     s <= x;
37.
38. end;

```

- **Entité** : Elle définit l'horloge système clk_50, une entrée de réinitialisation reset (bouton poussoir) et une sortie s (LED).
- **Architecture** : Le fonctionnement repose sur deux processus :
 1. **Division de fréquence** : Un premier processus incrémente un vecteur timer de 24 bits (23 downto 0) à chaque front de l'horloge 50 MHz. Le bit de poids fort (timer(23)) est extrait pour créer une horloge lente nommée clk.
 2. **Clignotement** : Un second processus utilise cette horloge lente pour inverser l'état d'un signal interne x ($x \leq \text{not}(x)$), créant le clignotement final assigné à la sortie s.

Une fois le code saisi, nous avons lancé le processus de Full Compilation pour générer la Netlist.

b) Affectation des Broches (Pin Planner)

Pour que le circuit fonctionne physiquement sur la carte, nous avons lié les signaux aux composants via le Pin Planner.

Node Name	Direction	Location	I/O Bank	VREF Group	I/O Standard	Reserved	Current Strength	Slew Rate
clk_50	Input	PIN_Y2	2	B2_N0	2.5 V (default)		8mA (default)	
reset	Input	PIN_M23	6	B6_N2	2.5 V (default)		8mA (default)	
s	Output	PIN_E21	7	B7_N0	2.5 V (default)		8mA (default)	2 (default)

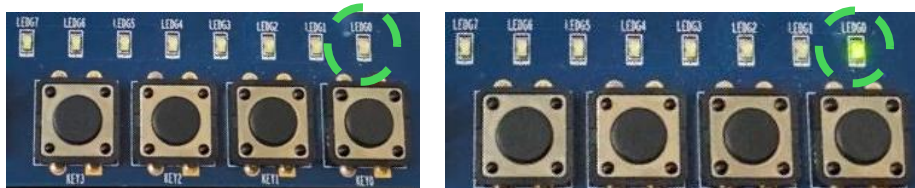
Nom du Signal	Type	Broche (Pin)	Composant Physique
clk_50	Entrée	PIN_Y2	Horloge 50 MHz
reset	Entrée	PIN_M23	Bouton Poussoir
s	Sortie	PIN_E21	Diode LED (LEDG0)

c) Programmation

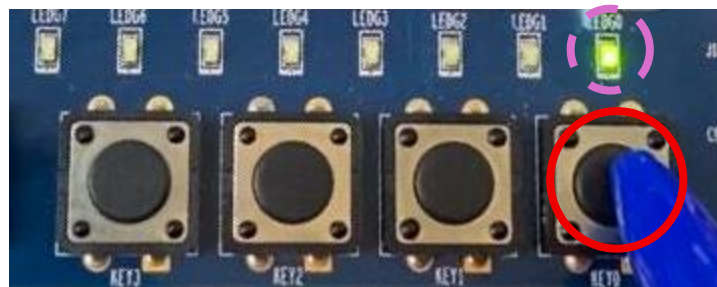
La dernière étape a consisté à ouvrir le Programmer. Nous avons vérifié que le matériel USB-Blaster était détecté, puis nous avons chargé le fichier de programmation .sof dans le FPGA.

Résultats et Observations : En manipulant le bouton poussoir, nous avons observé deux comportements distincts :

1. **Bouton relâché (reset = '1') :** La LED connectée à la broche E21 clignote lentement. Le diviseur de fréquence fonctionne normalement, le signal clk oscille et entraîne l'inversion du signal x.



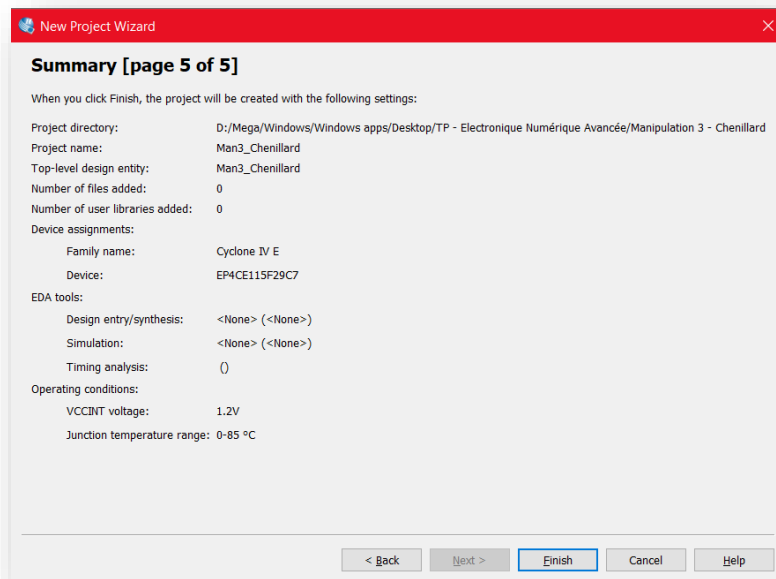
2. **Bouton maintenu appuyé (reset = '0') :** Comme défini dans le code VHDL (if reset = '0' then x <= '1'), le signal x est forcé à l'état haut. Simultanément, le timer est remis à zéro. Par conséquent, **la LED s'arrête de clignoter et reste allumée en continu** tant que l'appui est maintenu.



III. Manipulation 3 : Le Chenillard

L'objectif de cette manipulation était de réaliser un "chenillard", c'est-à-dire une animation lumineuse où une LED active se déplace cycliquement le long d'un vecteur de 4 LEDs. Cela a nécessité l'utilisation d'un registre à décalage (rotation) couplé à un diviseur de fréquence pour contrôler la vitesse de l'animation.

Nous avons créé un nouveau projet nommé Man3_Chenillard pour cette tâche .



a) Conception et Explication du Code VHDL

Pour cette manipulation, nous avons utilisé le code suivant :

```
1. library ieee;
2. use ieee.std_logic_1164.all;
3. use ieee.std_logic_unsigned.all;
4. use ieee.numeric_std.all;
5.
6. entity Man3_Chenillard is
7. port(
8.     clk_50, reset : in std_logic;
9.     leds : out std_logic_vector(0 to 3)
10.);
11. end;
12.
13. architecture behavior of Man3_Chenillard is
14. signal clk: std_logic := '0';
15. signal div : std_logic_vector(23 downto 0);
16. signal A : std_logic_vector(3 downto 0);
17.
```

```

18. begin
19. process(clk_50, reset)
20.     begin
21.         if reset = '0' then
22.             div <= (others => '0');
23.         elsif rising_edge(clk_50) then
24.             div <= div + 1;
25.         end if;
26.     end process;
27. clk <= div(23);
28.
29. process(clk, reset)
30.     begin
31.         if reset = '0' then
32.             A <= "0001";
33.         elsif rising_edge(clk) then
34.             sA <= A(2 downto 0) & A(3);
35.         end if;
36.     end process;
37. leds <= A;
38. end;

```

Entité : Elle comporte l'horloge système clk_50, un bouton de réinitialisation reset (actif à l'état bas), et un vecteur de sortie leds de 4 bits (de 0 à 3) pour piloter les lumières.

Architecture : Le comportement repose sur deux blocs logiques :

1. **Diviseur de fréquence :** Un compteur div de 24 bits génère une horloge lente clk à partir du bit de poids fort (div(23)), ramenant la fréquence de 50 MHz à quelques Hertz.

2. **Rotation (Chenillard):**

- À l'initialisation (reset = '0'), le vecteur interne A est forcé à "0001" (LED 1 allumée).
- À chaque front montant de l'horloge lente, l'instruction `A <= A(2 downto 0) & A(3);` effectue une rotation vers la gauche des bits. Le bit de poids fort est réinjecté en position de poids faible.
- La séquence obtenue est donc : 0001 → 0010 → 0100 → 1000 → 0001...

b) Affectation des Broches (Pin Planner)

Pour visualiser le résultat, nous avons assigné le vecteur de sortie aux 4 premières LEDs vertes (LEDG) de la carte FPGA.

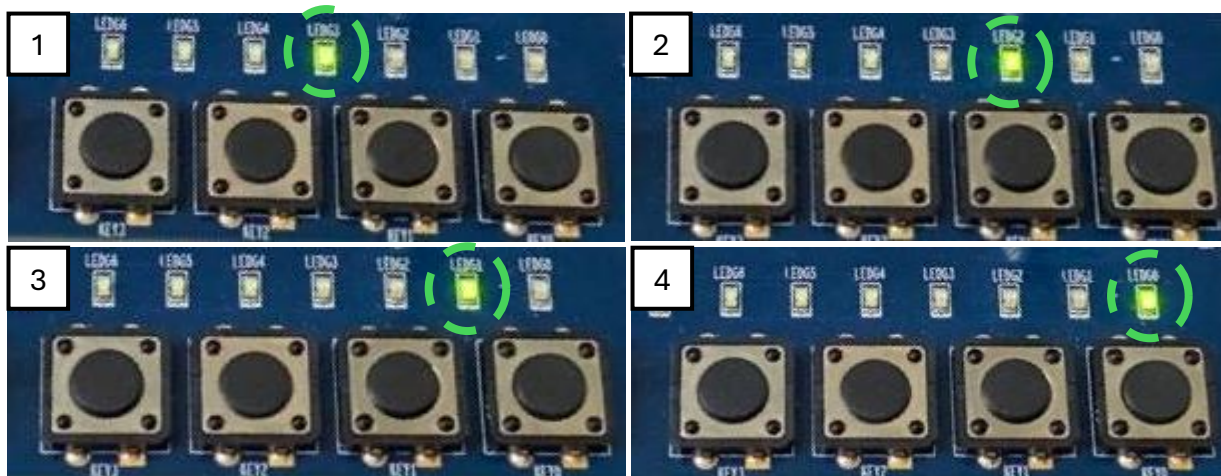
Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate
clk_50	Input	PIN_Y2	2	B2_N0	PIN_J1	2.5 V (default)		8mA (default)	
leds[0]	Output	PIN_E21	7	B7_N0	PIN_H3	2.5 V (default)		8mA (default)	2 (default)
leds[1]	Output	PIN_E22	7	B7_N0	PIN_G3	2.5 V (default)		8mA (default)	2 (default)
leds[2]	Output	PIN_E25	7	B7_N1	PIN_G4	2.5 V (default)		8mA (default)	2 (default)
leds[3]	Output	PIN_E24	7	B7_N1	PIN_H4	2.5 V (default)		8mA (default)	2 (default)

Nom du Signal	Type	Broche (Pin)	Composant Physique
clk_50	Entrée	PIN_Y2	Horloge 50 MHz
reset	Entrée	PIN_M23	Bouton Poussoir
leds[0]	Sortie	PIN_E21	LEDG0
leds[1]	Sortie	PIN_E22	LEDG1
leds[2]	Sortie	PIN_E25	LEDG2
leds[3]	Sortie	PIN_E24	LEDG3

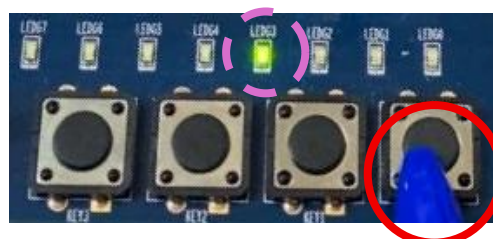
c) Programmation

Après avoir généré le fichier .sof et programmé la carte via l'USB-Blaster :

- **Observation :** Nous avons constaté que la lumière se déplaçait cycliquement de la LEDG0 à la LEDG3.



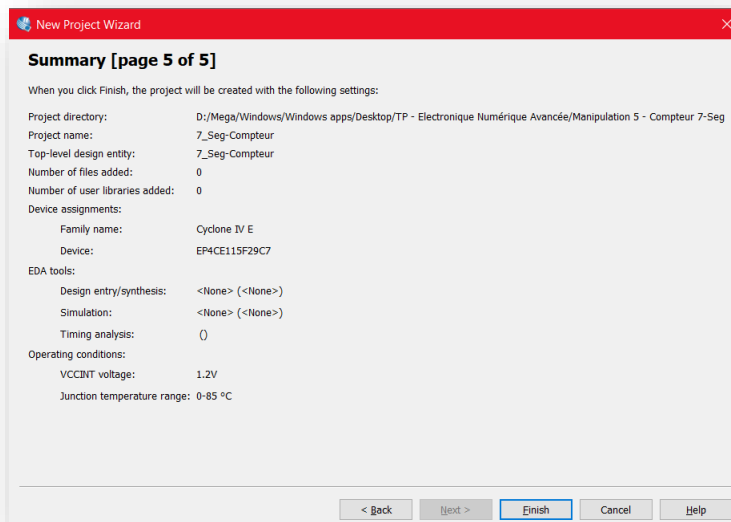
- **Reset :** Un appui sur le bouton reset fige la séquence et force l'allumage de la première LED (état initial "0001"), confirmant la logique de réinitialisation asynchrone décrite dans le code.



IV. Manipulation 4 : Compteur/Décompteur 7-Segments

L'objectif de cette manipulation était d'afficher la valeur d'un compteur sur un afficheur 7-segments de la carte FPGA. Contrairement à l'affichage binaire sur LEDs, cela permet une lecture plus naturelle (hexadécimale) pour l'utilisateur. De plus, nous avons ajouté une logique de contrôle permettant d'inverser le sens du comptage.

Nous avons créé un projet nommé Afficheur_Compteur pour cette tâche.



a) Conception et Explication du Code VHDL

Nous avons utilisé le code suivant, qui intègre le diviseur d'horloge, la logique de comptage bidirectionnelle et le décodeur 7-segments :

```
1. library ieee;
2. use ieee.std_logic_1164.all;
3. use ieee.std_logic_unsigned.all;
4. use ieee.numeric_std.all;
5.
6. entity Afficheur_Compteur is
7. port(
8.     clk_50, reset, mode : in std_logic;
9.     Afficheur : out std_logic_vector(6 downto 0)
10. );
11. end;
12.
13. architecture behavior of Afficheur_Compteur is
14. signal clk: std_logic := '0';
15. signal div : std_logic_vector(23 downto 0);
16. signal A : std_logic_vector(3 downto 0);
17.
```

```

18. begin
19. process(clk_50, reset)
20.     begin
21.         if reset = '0' then
22.             div <= (others => '0');
23.         elsif rising_edge(clk_50) then
24.             div <= div + 1;
25.         end if;
26.     end process;
27. clk <= div(23);
28.
29. process(clk, reset)
30.     begin
31.         if reset = '0' then
32.             A <= "0000";
33.         elsif rising_edge(clk) then
34.             if mode = '1' then
35.                 A <= A + 1;
36.             else
37.                 A <= A - 1;
38.             end if;
39.         end if;
40.     end process;
41.
42.     Afficheur <= "0000001" when A = "0000" else -- 0
43.     "1001111" when A = "0001" else -- 1
44.     "0010010" when A = "0010" else -- 2
45.     "0000110" when A = "0011" else -- 3
46.     "1001100" when A = "0100" else -- 4
47.     "0100100" when A = "0101" else -- 5
48.     "0100000" when A = "0110" else -- 6
49.     "0001111" when A = "0111" else -- 7
50.     "0000000" when A = "1000" else -- 8
51.     "0000100" when A = "1001" else -- 9
52.     "0001000" when A = "1010" else -- A
53.     "1100000" when A = "1011" else -- b
54.     "0110001" when A = "1100" else -- C
55.     "1000010" when A = "1101" else -- d
56.     "0110000" when A = "1110" else -- E
57.     "0111000" when A = "1111"; -- F
58. end;

```

Entité : Elle ajoute une entrée mode par rapport aux manipulations précédentes.

Architecture :

1. **Diviseur :** Génère une horloge lente clk à partir de clk_50 pour que le changement de valeur soit visible (environ 3 Hz).
2. **Compteur/Décompteur :** Ce processus vérifie l'état de l'entrée mode à chaque front montant de l'horloge lente :

- Si mode = '1', le signal interne A est incrémenté ($A + 1$).
- Si mode = '0', le signal interne A est décrémenté ($A - 1$).

3. **Transcodeur 7-Segments** : Une logique combinatoire convertit la valeur de 4 bits A en un vecteur de 7 bits pour piloter les segments. La logique est inversée (0 pour allumer), ce qui correspond à un afficheur à **Anode Commune**.

b) Affectation des Broches (Pin Planner)

Les broches ont été configurées pour utiliser l'horloge système, un bouton poussoir pour le reset, un interrupteur pour le mode, et l'afficheur HEX0.

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate
out Afficheur[6]	Output	PIN_G18	7	B7_N2	PIN_G18	2.5 V (default)		8mA (default)	2 (default)
out Afficheur[5]	Output	PIN_F22	7	B7_N0	PIN_F22	2.5 V (default)		8mA (default)	2 (default)
out Afficheur[4]	Output	PIN_E17	7	B7_N2	PIN_E17	2.5 V (default)		8mA (default)	2 (default)
out Afficheur[3]	Output	PIN_L26	6	B6_N1	PIN_L26	2.5 V (default)		8mA (default)	2 (default)
out Afficheur[2]	Output	PIN_L25	6	B6_N1	PIN_L25	2.5 V (default)		8mA (default)	2 (default)
out Afficheur[1]	Output	PIN_J22	6	B6_N0	PIN_J22	2.5 V (default)		8mA (default)	2 (default)
out Afficheur[0]	Output	PIN_H22	6	B6_N0	PIN_G19	2.5 V (default)		8mA (default)	2 (default)
in clk_50	Input	PIN_Y2	2	B2_N0	PIN_Y2	2.5 V (default)		8mA (default)	
in reset	Input	PIN_M23	6	B6_N2	PIN_M23	2.5 V (default)		8mA (default)	

Nom du Signal	Type	Broche (Pin)	Composant Physique
clk_50	Entrée	PIN_Y2	Horloge 50 MHz
reset	Entrée	PIN_M23	Bouton Poussoir (KEY0)
mode	Entrée	PIN_AB28	Interrupteur (SW0)
Afficheur[0] (a)	Sortie	PIN_G18	Segment a (HEX0)
Afficheur[1] (b)	Sortie	PIN_F22	Segment b (HEX0)
Afficheur[2] (c)	Sortie	PIN_E17	Segment c (HEX0)
Afficheur[3] (d)	Sortie	PIN_L26	Segment d (HEX0)
Afficheur[4] (e)	Sortie	PIN_L25	Segment e (HEX0)
Afficheur[5] (f)	Sortie	PIN_J22	Segment f (HEX0)
Afficheur[6] (g)	Sortie	PIN_H22	Segment g (HEX0)

c) Programmation

Après avoir chargé le fichier .sof sur la carte FPGA, nous avons effectué les tests suivants :

1. **Comptage** : Avec l'interrupteur mode (SW0) en position haute ('1'), l'afficheur incrémente : 0, 1, 2 ... F, 0.



2. **Décomptage** : En basculant l'interrupteur mode en position basse ('0'), le sens s'inverse immédiatement : F, E, D ... 0, F.



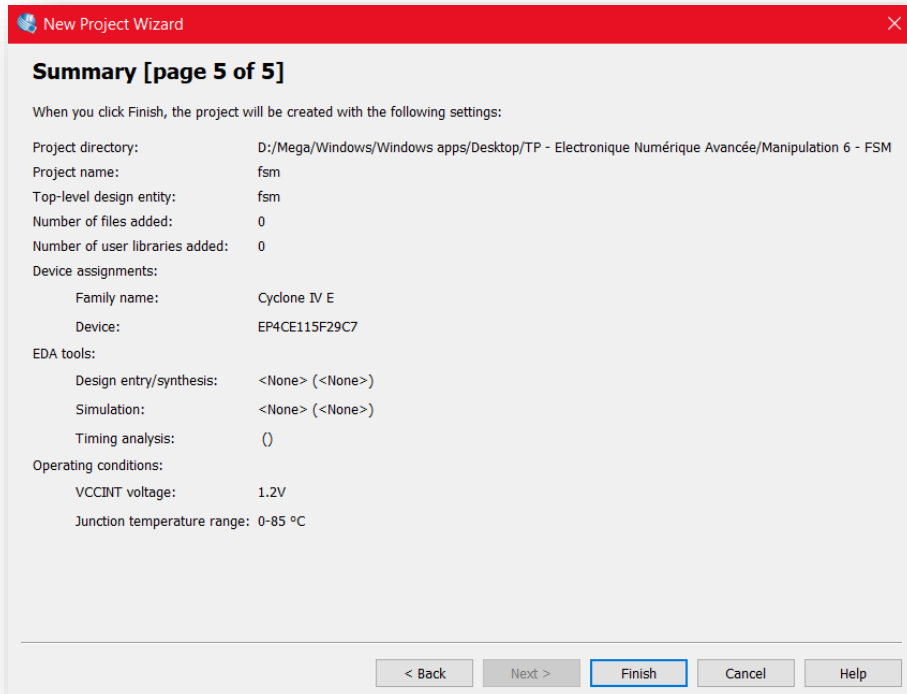
3. **Reset** : L'appui sur le bouton poussoir remet instantanément l'affichage à 0.



V. Manipulation 5 : FSM - Gestion d'un Carrefour

L'objectif de cette dernière manipulation était de modéliser le contrôleur de feux tricolores d'un carrefour routier (Axe 1 et Axe 2) en utilisant une Machine à États Finis (FSM). Le système doit gérer un cycle temporel précis et une priorité pour les piétons.

Nous avons créé un projet nommé fsm.



a) Conception et Explication du Code VHDL

1. Ajustement de l'Horloge

La carte FPGA dispose d'une horloge de 50 MHz ($T = 20$ ns), ce qui est trop rapide pour une perception humaine. Nous avons besoin d'un cycle d'environ 1 seconde (1 Hz).

Pour obtenir cette fréquence, nous avons utilisé un diviseur de fréquence (compteur). La valeur de division la plus proche est 2^{25} :

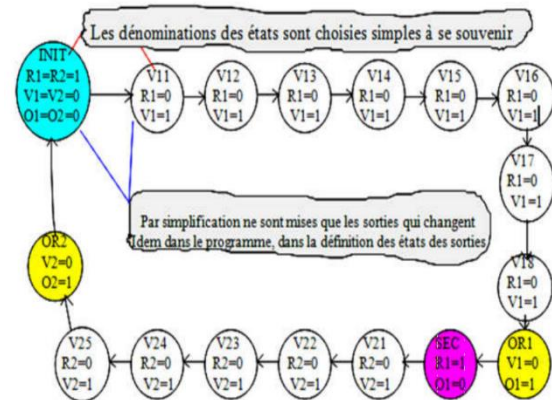
$$\frac{50 \times 10^6}{2^{25}} \approx \frac{50\,000\,000}{33\,554\,432} \approx 1.49 \text{ Hz}$$

Ceci nous donne un cycle proche de la seconde, suffisant pour l'observation visuelle.

2. Machine à États

Le code doit décrire un cycle complet de 17 étapes (E0 à E16), correspondant approximativement à 17 secondes :

- **Initialisation / Sécurité (E0)** : Les deux axes sont au Rouge.
- **Vert Axe 1 (E1 à E8)** : Durée de 8 "secondes".
- **Orange Axe 1 (E9)** : Durée de 1 "seconde".
- **Sécurité (E10)** : Rouge général pendant 1 "seconde".
- **Vert Axe 2 (E11 à E15)** : Durée de 5 "secondes".
- **Orange Axe 2 (E16)** : Durée de 1 "seconde".



3. Gestion Piéton

Une logique d'interruption a été ajoutée. Si le bouton piéton (ap1) est actionné pendant que le feu de l'Axe 1 est vert (états E1 à E6), la machine force une transition immédiate vers l'état de fin de cycle vert (vers E10), réduisant ainsi le temps d'attente pour les piétons.

4. Le code final

```

1. library ieee;
2. use ieee.std_logic_1164.all;
3. use ieee.std_logic_unsigned.all;
4. use ieee.numeric_std.all;
5.
6. entity fsm is
7. port(
8.     clk_50, reset, ap1 : in std_logic;
9.     r1, r2, v1, v2, o1, o2 : out std_logic
10. );
11. end fsm;
12.
13. architecture behavior of fsm is
14.
15.     signal clk : std_logic := '0';
16.     signal div : std_logic_vector(25 downto 0);
17.
18.     type etat17 is (E0, E1, E2, E3, E4, E5, E6, E7, E8, E9, E10, E11, E12, E13, E14, E15, E16);
19.     signal etat_present, etat_future : etat17;
20.
21. begin
22.
23.     process(clk_50, reset)
24.     begin
25.         if reset = '0' then
26.             div <= (others => '0');
27.         elsif rising_edge(clk_50) then
28.             div <= div + 1;

```

```

29.         end if;
30.     end process;
31.
32.     clk <= div(25);
33.
34.     process(clk, reset)
35.     begin
36.         if reset = '0' then
37.             etat_present <= E0;
38.         elsif rising_edge(clk) then
39.             etat_present <= etat_future;
40.         end if;
41.     end process;
42.
43.     process(etat_present, ap1)
44.     begin
45.         case etat_present is
46.
47.             when E0 =>
48.                 r1 <= '0'; r2 <= '0';
49.                 v1 <= '1'; v2 <= '1';
50.                 o1 <= '1'; o2 <= '1';
51.                 etat_future <= E1;
52.
53.             when E1 =>
54.                 r1 <= '1'; r2 <= '0';
55.                 v1 <= '0'; v2 <= '1';
56.                 o1 <= '1'; o2 <= '1';
57.                 if ap1 = '1' then
58.                     etat_future <= E10;
59.                 else
60.                     etat_future <= E2;
61.                 end if;
62.
63.             when E2 =>
64.                 r1 <= '1'; r2 <= '0';
65.                 v1 <= '0'; v2 <= '1';
66.                 o1 <= '1'; o2 <= '1';
67.                 if ap1 = '1' then
68.                     etat_future <= E10;
69.                 else
70.                     etat_future <= E3;
71.                 end if;
72.
73.             when E3 =>
74.                 r1 <= '1'; r2 <= '0';
75.                 v1 <= '0'; v2 <= '1';
76.                 o1 <= '1'; o2 <= '1';
77.                 if ap1 = '1' then
78.                     etat_future <= E10;
79.                 else
80.                     etat_future <= E4;
81.                 end if;
82.
83.             when E4 =>
84.                 r1 <= '1'; r2 <= '0';

```

```

85.         v1 <= '0'; v2 <= '1';
86.         o1 <= '1'; o2 <= '1';
87.         if ap1 = '1' then
88.             etat_future <= E10;
89.         else
90.             etat_future <= E5;
91.         end if;
92.
93.     when E5 =>
94.         r1 <= '1'; r2 <= '0';
95.         v1 <= '0'; v2 <= '1';
96.         o1 <= '1'; o2 <= '1';
97.         if ap1 = '1' then
98.             etat_future <= E10;
99.         else
100.            etat_future <= E6;
101.        end if;
102.
103.    when E6 =>
104.        r1 <= '1'; r2 <= '0';
105.        v1 <= '0'; v2 <= '1';
106.        o1 <= '1'; o2 <= '1';
107.        if ap1 = '1' then
108.            etat_future <= E10;
109.        else
110.            etat_future <= E7;
111.        end if;
112.
113.    when E7 =>
114.        r1 <= '1'; r2 <= '0';
115.        v1 <= '0'; v2 <= '1';
116.        o1 <= '1'; o2 <= '1';
117.        etat_future <= E8;
118.
119.    when E8 =>
120.        r1 <= '1'; r2 <= '0';
121.        v1 <= '0'; v2 <= '1';
122.        o1 <= '1'; o2 <= '1';
123.        etat_future <= E9;
124.
125.    when E9 =>
126.        r1 <= '1'; r2 <= '0';
127.        v1 <= '1'; v2 <= '1';
128.        o1 <= '0'; o2 <= '1';
129.        etat_future <= E10;
130.
131.    when E10 =>
132.        r1 <= '0'; r2 <= '0';
133.        v1 <= '1'; v2 <= '1';
134.        o1 <= '1'; o2 <= '1';
135.        etat_future <= E11;
136.
137.    when E11 =>
138.        r1 <= '0'; r2 <= '1';
139.        v1 <= '1'; v2 <= '0';
140.        o1 <= '1'; o2 <= '1';

```

```

141.          etat_future <= E12;
142.
143.      when E12 =>
144.          r1 <= '0'; r2 <= '1';
145.          v1 <= '1'; v2 <= '0';
146.          o1 <= '1'; o2 <= '1';
147.          etat_future <= E13;
148.
149.      when E13 =>
150.          r1 <= '0'; r2 <= '1';
151.          v1 <= '1'; v2 <= '0';
152.          o1 <= '1'; o2 <= '1';
153.          etat_future <= E14;
154.
155.      when E14 =>
156.          r1 <= '0'; r2 <= '1';
157.          v1 <= '1'; v2 <= '0';
158.          o1 <= '1'; o2 <= '1';
159.          etat_future <= E15;
160.
161.      when E15 =>
162.          r1 <= '0'; r2 <= '1';
163.          v1 <= '1'; v2 <= '0';
164.          o1 <= '1'; o2 <= '1';
165.          etat_future <= E16;
166.
167.      when E16 =>
168.          r1 <= '0'; r2 <= '1';
169.          v1 <= '1'; v2 <= '1';
170.          o1 <= '1'; o2 <= '0';
171.          etat_future <= E0;
172.
173.      end case;
174.  end process;
175.
176. end behavior;
177.

```

b) Affectation des Broches (Pin Planner)

Les sorties de la FSM ont été connectées aux segments des afficheurs **HEX0** (Axe 1) et **HEX1** (Axe 2). Nous avons mappé les feux aux segments horizontaux pour simuler un feu tricolore :

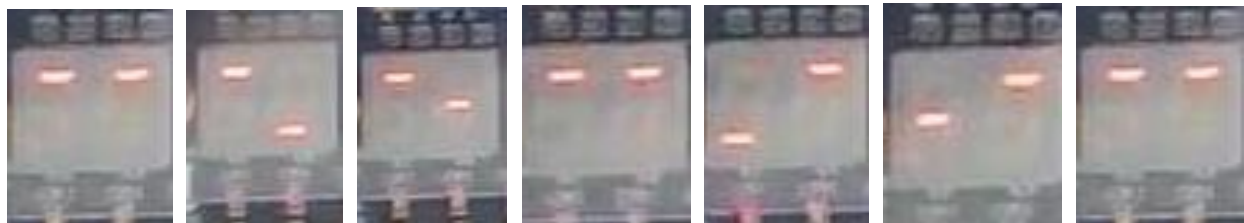
Node Name	Direction	Location	I/O Bank	VREF Group	Filter Location	I/O Standard	Reserved	Current Strength	Slew Rate
ap1	Input	PIN_M21	6	B6_N1	PIN_R22	2.5 V (default)		8mA (default)	
clk_50	Input	PIN_Y2	2	B2_N0	PIN_Y2	2.5 V (default)		8mA (default)	
o1	Output	PIN_H22	6	B6_N0	PIN_H22	2.5 V (default)		8mA (default)	2 (default)
o2	Output	PIN_U24	5	B5_N0	PIN_U24	2.5 V (default)		8mA (default)	2 (default)
r1	Output	PIN_G18	7	B7_N2	PIN_G18	2.5 V (default)		8mA (default)	2 (default)
r2	Output	PIN_M24	6	B6_N2	PIN_M24	2.5 V (default)		8mA (default)	2 (default)
reset	Input	PIN_M23	6	B6_N2	PIN_M23	2.5 V (default)		8mA (default)	
v1	Output	PIN_L26	6	B6_N1	PIN_L26	2.5 V (default)		8mA (default)	2 (default)
v2	Output	PIN_W22	5	B5_N0	PIN_W22	2.5 V (default)		8mA (default)	2 (default)

- **Rouge** → Segment du haut (a)
- **Orange** → Segment du milieu (g)
- **Vert** → Segment du bas (d)

Signal	Axe	Couleur/Fonction	Broche FPGA (DE2-115)	Segment Visé
r1	Axe 1	Rouge	PIN_G18	HEX0 (a)
o1	Axe 1	Orange	PIN_H22	HEX0 (g)
v1	Axe 1	Vert	PIN_L26	HEX0 (d)
r2	Axe 2	Rouge	PIN_M24	HEX1 (a)
o2	Axe 2	Orange	PIN_L24	HEX1 (g)
v2	Axe 2	Vert	PIN_W22	HEX1 (d)
ap1	-	Bouton Piéton	PIN_M21	KEY1

c) Programmation

L'utilisation des deux afficheurs 7-segments a rendu la vérification très intuitive.



1. Nous avons observé l'alternance des feux entre l'afficheur de droite (Axe 1) et celui de gauche (Axe 2).
2. La "barre" lumineuse descendait bien du haut (Rouge) vers le bas (Vert) en passant par le milieu (Orange).
3. L'appui sur le bouton piéton ap1 a bien interrompu le cycle vert de l'Axe 1 pour passer à l'orange, validant la gestion des priorités.

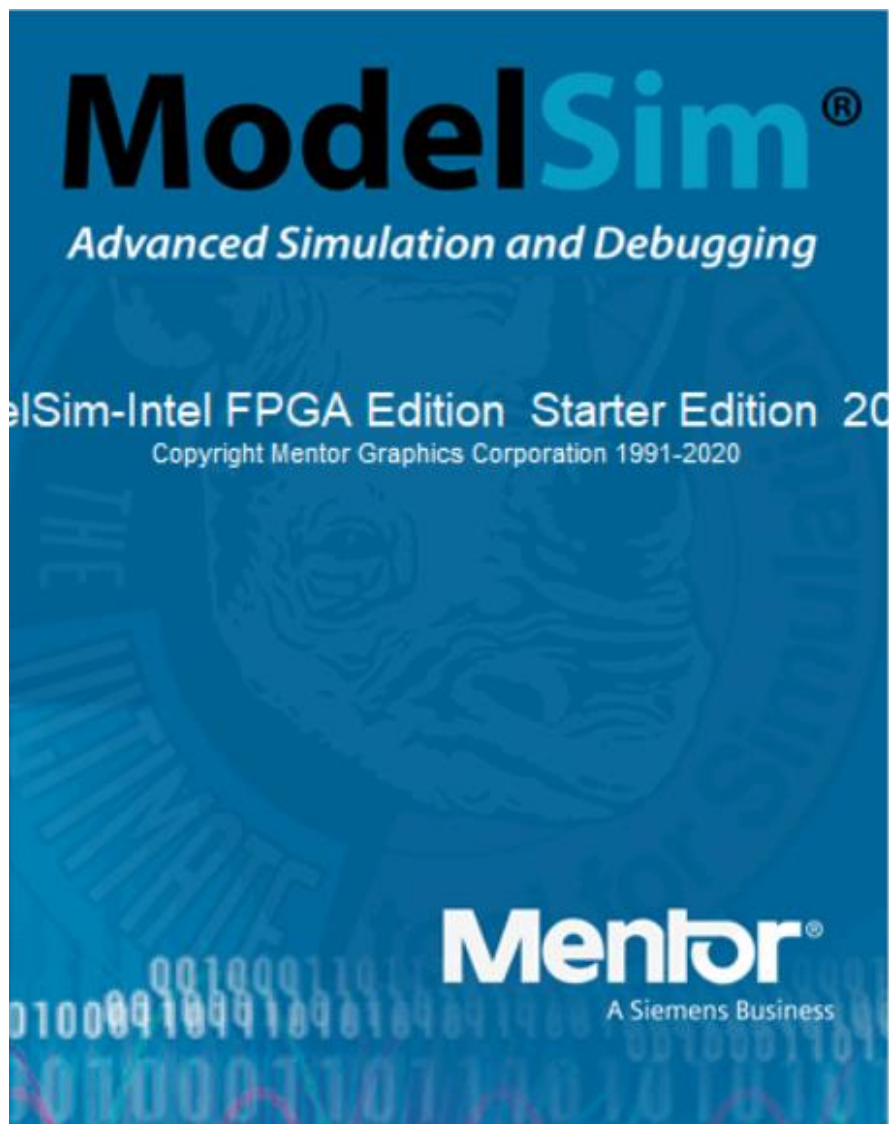
Conclusion Partielle

Cette première phase nous a permis de faire le lien entre la théorie et la pratique. Nous avons compris qu'écrire du code VHDL ne suffit pas : il faut savoir le compiler, l'assigner aux bonnes broches et le charger sur la carte.

Nous avons surtout retenu l'importance de la **division d'horloge**. Sans ralentir les 50 MHz de la carte, impossible de voir nos LEDs clignoter ou notre feu tricolore fonctionner. Voir nos codes (XOR, Compteurs, FSM) s'exécuter physiquement sur la carte FPGA a été la meilleure façon de valider notre compréhension du matériel.

DEUXIÈME PARTIE
SIMULATION COMPORTEMENTALE ET
VALIDATION TEMPORELLE

MODELSIM ALTERA



Introduction

Après avoir effectué la synthèse et l'implémentation physique sur la carte FPGA, il est essentiel de valider le fonctionnement théorique de nos circuits. La simulation permet de vérifier le comportement temporel et logique de nos descriptions VHDL sans les contraintes matérielles.

Dans cette partie, nous avons utilisé le logiciel **ModelSim** pour visualiser les chronogrammes de nos différentes entités.

Manipulation Unique : Registre à Décalage 4-bits (PISO)

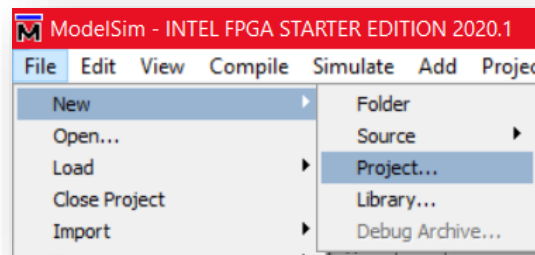
Dans cette seconde partie, nous avons utilisé l'outil de simulation **ModelSim** pour valider le comportement temporel d'un circuit séquentiel sans passer par l'implémentation physique. Le circuit étudié est un **registre à décalage de 4 bits** capable de charger une donnée en parallèle et de la sortir en série (PISO - Parallel In Serial Out) par décalage vers la droite.

1. Configuration de l'Environnement et du Projet

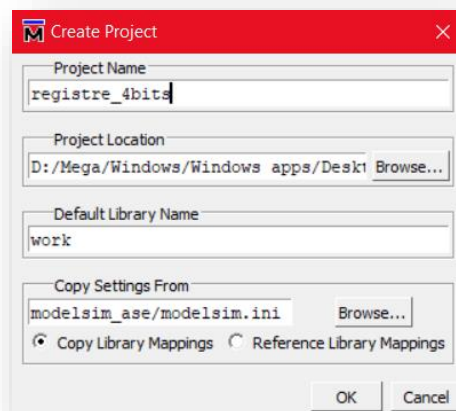
a. Création du Projet de Simulation

Pour réaliser cette simulation, nous avons suivi la procédure suivante directement dans ModelSim :

- Nous avons lancé ModelSim et accédé au menu **File > New > Project**.



- Nous avons défini le nom de projet et le dossier de travail.

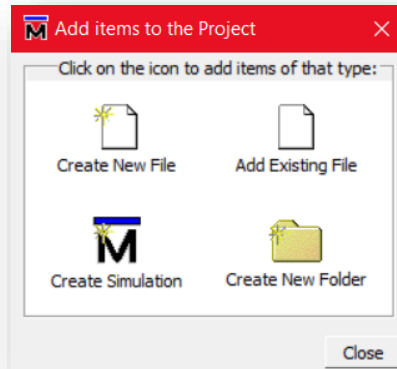


- La bibliothèque par défaut (Default Library Name) a été laissée sur work.

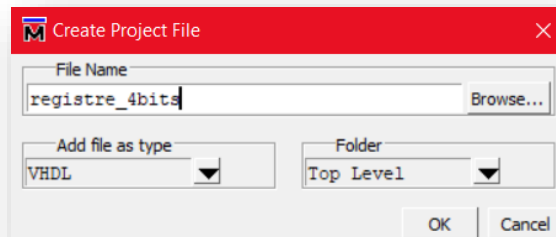
b. Création et Ajout des Fichiers VHDL

Une fois le projet créé, une boîte de dialogue "Add items to the Project" s'ouvre :

- Nous avons sélectionné **Create New File**.



- Nous avons spécifié le nom du fichier et choisi le type **VHDL**.



- Et plus tard, nous avons ajouté le fichier VHDL pour le banc d'essai.



- C'est dans ces fichiers que nous avons copié le code de nos entités et de nos architectures.

2. Compilation

Vérification de la syntaxe via *Compile All*. La validation (coches vertes) nous a permis de passer à la phase de simulation.

Compile	▶	Compile Selected			
Add to Project	▶	Compile All			
Remove from Project		Compile Out-of-Date			

	registre_4bits.vhd		VHDL	0	12/27/2025 01:04:36 am
	tb_registre_4bits.vhd		VHDL	1	12/27/2025 02:05:24 am

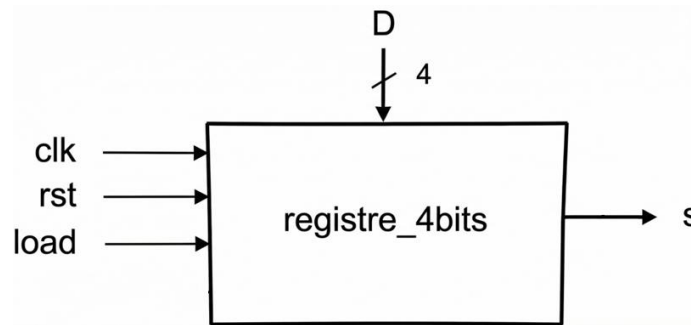
3. Analyse du Code VHDL

Code VHDL de l'entité registre_4bits : Ce code décrit un système synchrone sensible au front montant de l'horloge (clk) et doté d'un reset asynchrone (rst).

```
1. LIBRARY IEEE;
2. USE IEEE.std_logic_1164.all;
3.
4. entity registre_4bits is
5. port(
6.     clk , rst , load : in std_logic ;
7.     d : in std_logic_vector(3 downto 0);
8.     s : out std_logic
9. );
10. end entity ;
11.
12. architecture arch of registre_4bits is
13.     signal x : std_logic_vector(3 downto 0 );
14.
15. begin
16.     process(clk , rst )
17.     begin
18.         if (rst = '1') then
19.             x <= "0000"; -- Remise à zéro asynchrone
20.         elsif rising_edge(clk) then
21.             if (load='1') then
22.                 x <= d ; -- Chargement parallèle
23.             else
24.                 x <= '0' & x(3 downto 1) ; -- Décalage logique à droite
25.             end if ;
26.         end if ;
27.     end process ;
28.
29.     s <= x(0); -- Sortie série (LSB)
30.
31. end arch ;
32.
```

Entrées/Sorties :

- d (4 bits) : Entrée de données parallèle.
- load : Signal de commande. Si '1', on charge d. Si '0', on décale.
- s : Sortie série (LSB - Bit de poids faible).



Fonctionnement:

- Si load = '1', le registre charge instantanément la valeur présente sur l'entrée d.
- Si load = '0', le contenu du registre se décale vers la droite. Le bit de poids faible x(0) sort sur s, et un '0' est inséré en tête (MSB).

4. Analyse du Testbench

Pour automatiser la vérification, nous avons créé un second fichier VHDL nommé *tb_registre_4bits.vhd*. Ce "Testbench" n'a pas d'entrées physiques mais génère virtuellement les signaux pour stimuler le composant.

Code VHDL du Testbench :

```
1. LIBRARY IEEE;
2. USE IEEE.std_logic_1164.all;
3. USE work.all;
4.
5. entity tb_registre_4bits is
6. end entity;
7.
8. architecture tb of tb_registre_4bits is
9.
10.     component registre_4bits
11.     port(
12.         clk, rst, load : in std_logic;
13.         d : in std_logic_vector(3 downto 0);
14.         s : out std_logic
15.     );
16. end component;
17.
18. signal clk : std_logic := '0';
19. signal rst, load, s : std_logic := '0';
```

```

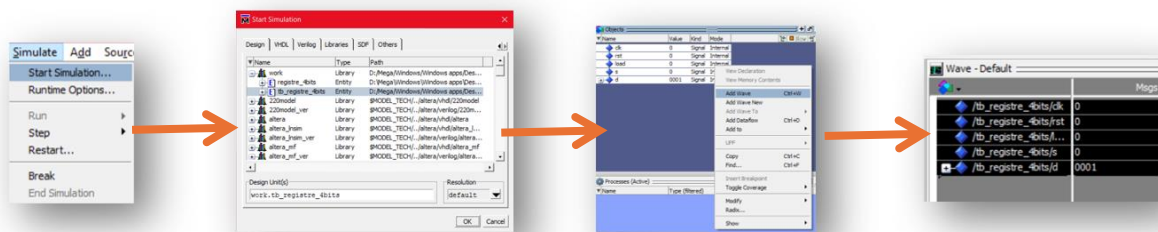
20.     signal d : std_logic_vector(3 downto 0) := (others => '0');
21.
22. begin
23.
24.     DUT: registre_4bits port map (
25.         clk => clk,
26.         rst => rst,
27.         load => load,
28.         d   => d,
29.         s   => s
30.     );
31.
32.     clk_process: process
33.     begin
34.         clk <= '0';
35.         wait for 10 ns;
36.         clk <= '1';
37.         wait for 10 ns;
38.     end process;
39.
40.     stim_proc: process
41.     begin
42.         rst <= '1';
43.         wait for 40 ns;
44.         rst <= '0';
45.         wait for 20 ns;
46.
47.         wait until falling_edge(clk);
48.         d <= "1010";
49.         load <= '1';
50.         wait for 20 ns;
51.
52.         load <= '0';
53.         wait for 80 ns;
54.
55.         d <= "1100";
56.         load <= '1';
57.         wait for 20 ns;
58.
59.         load <= '0';
60.         wait for 80 ns;
61.
62.         d <= "0001";
63.         load <= '1';
64.         wait for 20 ns;
65.
66.         load <= '0';
67.         wait for 80 ns;
68.
69.         wait;
70.     end process;
71.
72. end tb;
73.

```

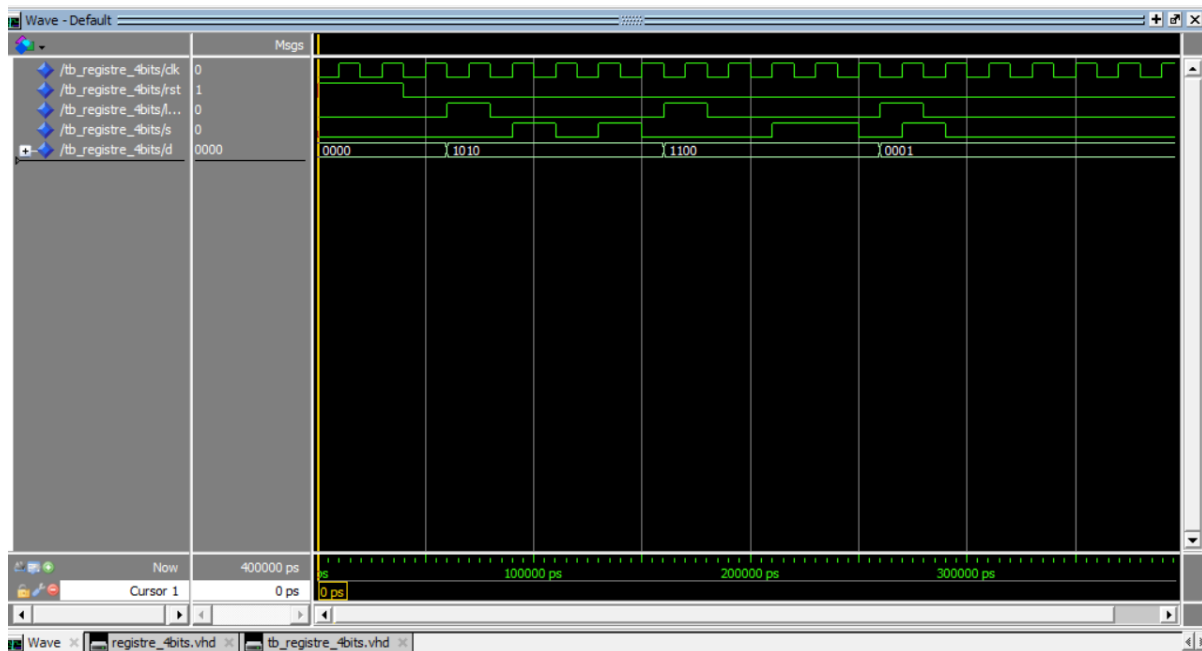
- **Génération d'Horloge** : Un processus génère une horloge clk de période **20 ns** (50 MHz).
- **Scénarios de Test** :
 1. **Initialisation** : Application du Reset pour partir d'un état connu.
 2. **Test 1 (Valeur "1010")** : Activation de load pour charger la valeur, puis désactivation pour observer la sortie série des bits un par un (0, puis 1, puis 0, puis 1).
 3. **Test 2 (Valeur "1100")** : Chargement de "1100" et observation du décalage.
 4. **Test 3 (Valeur "0001")** : Vérification du bit de poids faible.

5. Résultats de Simulation

Après avoir lancé la simulation (*Simulate > Start Simulation*) et ajouté les signaux au visualiseur (*Add to Wave*).



Nous avons observé les chronogrammes suivants :



1. **Chargement Parallèle** : Lorsque load est à '1' au front montant de l'horloge, le registre capture instantanément la valeur présente sur l'entrée d (testé avec "1010", "1100" et "0001").
2. **Conversion Série** : Lorsque load passe à '0', le décalage s'active. Durant les **4 cycles d'horloge suivants (80 ns)**, les bits se décalent et la sortie s affiche successivement les bits (du LSB au MSB).
3. **Validation** : La simulation de **400 ns** confirme que chaque vecteur d'entrée est correctement converti en un flux série synchronisé sur la sortie s.

Conclusion Partielle

Dans cette seconde partie, nous avons découvert l'intérêt de la simulation avant l'implémentation. Utiliser **ModelSim** nous a permis de vérifier le fonctionnement interne de notre registre à décalage (PISO) à la nanoseconde près, ce qui est impossible à l'œil nu sur la carte.

Nous avons bien saisi la différence entre le code du circuit et le **Testbench**. Ce dernier est essentiel pour générer des stimuli virtuels et s'assurer que notre logique est robuste avant même de toucher au FPGA.

Conclusion Générale

L'ensemble de ces séances de travaux pratiques en **Électronique Numérique Avancée** nous a offert une vision globale et pratique du métier de concepteur de systèmes numériques. Nous avons parcouru l'intégralité du "Design Flow" standard de l'industrie :

Spécification → Description VHDL → Simulation → Synthèse → Implémentation.

Ces acquis constituent une base solide pour aborder des projets de systèmes embarqués plus ambitieux, intégrant potentiellement des processeurs ou des interfaces de communication avancées.